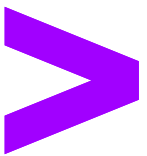


Emulating User Gestures by Actions



Module Objective

At the end of this course, you will be able to:

- Understand about actions class in WebDriver
- List keyboard and mouse event along with commonly used methods
- Perform automation of mouse events
- Perform automation of keyword events



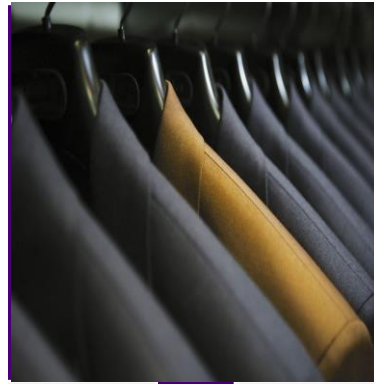
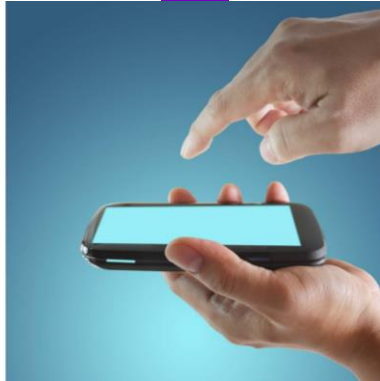
Ground Rules

For a successful class, please:



Arrive on time

Keep your cell phones off



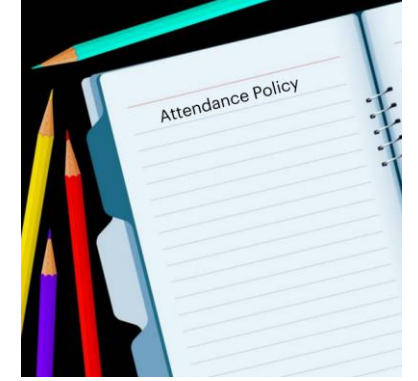
Wear business formals

Assist and show respect to your colleagues irrespective of skill and knowledge level



Do not use the web for personal reasons during training sessions

Adhere to attendance policy as directed by training coordinator



Emulating keyboard and mouse events using Actions API

- 01** Why we need Actions class
- 02** Overview of Actions class
- 03** Automate mouse events
- 04** Automate keyboard events

Why we need Actions class

Why we need Actions Class(1)

Need Of Actions class

- **Complex User Interactions:** It enables you to simulate complex actions like mouse movements, drag-and-drop, and right-clicking.
- **Keyboard Actions:** You can send key presses to an element, which is helpful for testing forms or keyboard shortcuts.
- **Chaining Actions:** The Actions class allows you to chain multiple actions together, which can make your tests more concise and easier to read.
- **Hovering:** It can simulate mouse hover events, which is useful for testing menus and dropdowns that appear on hover.
- **Customizable Actions:** You can customize the duration of actions and control the timing, which is helpful for mimicking real user behavior.

Why we need Actions class (2)

Understand Automation Scenario 01

- Sub-menu items are visible in the Document Object Model (DOM) when we mouse hover on the main menu
- Go to AccenSportz home page URL: <https://lkmdemoaut.accenture.com/AccenSportz/#/> and mouse hover on “Sports Coaching” on main menu.



Overview of Actions

Overview of Actions (1)

Overview of Actions API

- Actions are user interface API for imitating complex user gestures on the User Interface (UI) of the web application
- WebDriver has introduced advanced user-interactions API using Actions class

```
● build() : Action - Actions
● click() : Actions - Actions
● click(WebElement target) : Actions - Actions
● clickAndHold() : Actions - Actions
● clickAndHold(WebElement target) : Actions - Actions
● contextClick() : Actions - Actions
● contextClick(WebElement target) : Actions - Actions
● doubleClick() : Actions - Actions
● doubleClick(WebElement target) : Actions - Actions
● dragAndDrop(WebElement source, WebElement target) : Actions - Actions
● dragAndDropBy(WebElement source, int xOffset, int yOffset) : Actions - Actions
● equals(Object obj) : boolean - Object
● getClass() : Class<?> - Object
● hashCode() : int - Object
● keyDown(CharSequence key) : Actions - Actions
● keyDown(WebElement target, CharSequence key) : Actions - Actions
● keyUp(CharSequence key) : Actions - Actions
● keyUp(WebElement target, CharSequence key) : Actions - Actions
● moveByOffset(int xOffset, int yOffset) : Actions - Actions
● moveToElement(WebElement target) : Actions - Actions
● moveToElement(WebElement target, int xOffset, int yOffset) : Actions - Actions
● notify() : void - Object
● notifyAll() : void - Object
● pause(Duration duration) : Actions - Actions
● pause(long pause) : Actions - Actions
● perform() : void - Actions
● release() : Actions - Actions
● release(WebElement target) : Actions - Actions
● sendKeys(CharSequence... keys) : Actions - Actions
● sendKeys(WebElement target, CharSequence... keys) : Actions - Actions
● tick(Action action) : Actions - Actions
● tick(Interaction... actions) : Actions - Actions
● toString() : String - Object
● wait() : void - Object
● wait(long timeout) : void - Object
```

Overview of Actions (2)

Syntax for Creating Actions

- To handle the scenario explained in earlier slides, we can use Actions class in Selenium WebDriver
- The first step is to create object of an Actions class by passing the WebDriver instance as the argument in Actions constructor

```
Actions actions = new Actions(driver);  
actions.xxxx.xxxx.xxxx.build().perform();
```

- build() is required only when there are sequence of actions to be compiled in single step
- build() is not required when there is only single action
- perform() is mandatory to execute the actions

Automate Mouse Events

Automate Mouse Events (1)

Commonly Used Methods with Actions Mouse Events

- Actions class provide the commonly used methods to handle mouse events as shown below

Method Name	Method Description
doubleClick()	Helps to carry out a double-click at the current mouse location or element
clickAndHold()	Press (without releasing) at the current mouse location or element
release()	Releases the depressed left mouse button at the current mouse location or element
contextClick()	Carry out a right-click at the current mouse location or element
dragAndDrop(source, target)	Carry out press-and-hold at the source element location, shift to the target element location, then release the mouse



Automate Mouse Events (2)

Commonly Used Methods with Actions Mouse Events

- Actions class provide the commonly used methods to handle mouse events as shown below

Method Name	Method Description
dragAndDropBy(source, x-offset, y-offset)	Carry out press-and-hold at the source element location, shift to the target offset, then release the mouse
moveByOffset(x-offset, y-offset)	Moves the mouse from its current position (or 0,0) by the given offset
moveToElement(toElement)	Hovers/Shifts the mouse to the center of the element

Automate Mouse Events (3)

Demo: Automate a scenario to demonstrate Mouse Hover Event

1.

Open AccenSportz URL: <https://lkmdemoaut.accenture.com/AccenSportz/#/> on a web browser

2.

Maximize the browser window and use implicit wait command

3.

Mouse hover on sports coaching & click

4.

Close the browser

Automate Mouse Events (4)

Demo: Automate a scenario to demonstrate Mouse Hover Event

```
package actionsClass;
import java.time.Duration;

public class MouseOver {
    WebDriver driver;
    @BeforeClass
    public void setUp() {
        driver = new ChromeDriver();
        driver.manage().window().maximize();
        driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));
        driver.get("https://lkmdemoaut.accenture.com/AccenSportz/#/");
    }
    @Test
    public void testMouseHover() throws InterruptedException {
        // Locate the element to hover over
        WebElement hoverElement = driver.findElement(By.LinkText("Player & Team Strategy"));
        hoverElement.click();
        Thread.sleep(2000);
        // Create an Actions instance
        Actions actions = new Actions(driver);

        // Perform the hover action
        actions.moveToElement(driver.findElement(By.xpath("//*[@id=\"polls\"]/b")))
        .moveToElement(driver.findElement(By.xpath("//*[@id=\"menu3\"]/li[1]/ul/li[1]/a")))
        .click().build().perform();

    }
    @AfterClass
    public void tearDown() {
        // Close the browser
        driver.quit();
    }
}
```

Automate Mouse Events (5)

Demo: Automate a scenario to demonstrate Drag and Drop Event

1.

Open Test Me URL: <https://lkmdemoaut.accenture.com/AccenSportz/#/> on a web browser

2.

Maximize the browser window and use implicit wait command

3.

Go to Sports Fantasy and create a team

4.

Close the browser

Automate Mouse Events (6)

Code Snippet For drag and drop event

```
package actionsClass;

import java.time.Duration;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.interactions.Actions;
import org.testng.annotations.AfterTest;
import org.testng.annotations.BeforeTest;
import org.testng.annotations.Test;
```

```
}
@BeforeMethod
public void beforeMethod() {
    driver=new ChromeDriver();
    driver.manage().window().maximize();
    driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));
    driver.get("https://lkmdemoaut.accenture.com/AccenSportz/#/");
}
@AfterMethod
public void afterMethod() {
    driver.close();
}
}
```

Automate Mouse Events (6)

Code Snippet For drag and drop event

```
Run ALL
public class Drag_N_Drop_Demo {
    WebDriver driver;
    @Test
    Run / Debug
    public void f() {

        driver.findElement(By.id("fantasysports")).click();
        WebElement player1=driver.findElement
            (By.xpath("//div[@class='cdk-drop-list example-list']//div[2]"));
        WebElement player2=driver.findElement(By.xpath("//div[@class='cdk-drop-list example-list']//div[3]"));
        WebElement player3=driver.findElement(By.xpath("//div[@class='cdk-drop-list example-list']//div[4]"));
        WebElement player4=driver.findElement(By.xpath("//div[@class='cdk-drop-list example-list']//div[5]"));

        WebElement target=driver.findElement(By.id("cdk-drop-list-1"));
        Actions act=new Actions(driver);
        act.dragAndDrop(player1, target).build().perform();
        act.dragAndDrop(player2, target).build().perform();
        act.dragAndDrop(player3, target).build().perform();
        act.dragAndDrop(player4, target).build().perform();
        driver.findElement(By.id("submit1")).click();

    }
}
```

Automate Keyboard Events

Automate Keyboard Events (1)

Commonly Used Methods with Actions Keyboard Events

- Actions class provide the commonly used methods to handle keyboard events as shown below

Method Name	Method Description
keyDown(key)	Performs key press on a key from keyboard
keyDown(target, key)	Performs key press after focusing an element and select a key from keyboard
keyUp(key)	Performs key release on a key from keyboard
keyUp(target, key)	Performs key release after focusing an element and select a key from keyboard
sendKeys(onElement, Charsequence)	Communicates a series of keystrokes onto the element

Automate Keyboard Events (2)

Code Snippets for Keyboard Events

// Automate Keyboard Events

```
Actions actions = new Actions(driver);  
actions.keyDown(search, Keys.SHIFT).sendKeys("computer").keyUp(Keys.SHIFT).build().perform();
```

// Automate Keyboard + Mouse Events

```
Actions actions = new Actions(driver);  
actions.keyDown(Keys.SHIFT).click(signInLink).keyUp(Keys.SHIFT).build().perform();
```

Automate Keyboard Events (3)

Demo: Automate a scenario with Keyboard Events

1.

Open AccenSportz URL: <https://lkmdemoaut.accenture.com/AccenSportz/#/> on a web browser

2.

Maximize the browser window and use implicit wait command

3.

Click on sports coaching & select the slot

4.

Close the browser

Automate Keyboard Events (4)

Code Snippet keyboard event

```
package actionsClass;

import java.time.Duration;
import org.openqa.selenium.By;
import org.openqa.selenium.Keys;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.interactions.Actions;
import org.testng.annotations.AfterClass;
import org.testng.annotations.BeforeClass;
import org.testng.annotations.Test;
```

```
public class KeyboardEvents {

    WebDriver driver;
    @BeforeClass
    public void setUp() {
        driver = new ChromeDriver();
        driver.manage().window().maximize();
        driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));
        driver.get("https://lkmdemoaut.accenture.com/AccenSportz/#/");
    }
}
```

```
    @AfterClass
    public void tearDown() {
        // Close the browser
        driver.quit();
    }
}
```

Automate Keyboard Events (5)

Code Snippet for Keyboard Event

```
@Test
public void testMouseHover() throws InterruptedException {
    // Locate the element to hover over
    WebElement hoverElement = driver.findElement(By.xpath("/html/body/app-root/app-home/div/div/div[3]/div/div[1]/button"));

    // Create an Actions instance
    Actions actions = new Actions(driver);

    // Perform the hover action
    actions.moveToElement(hoverElement).perform();
    hoverElement.click();

    driver.findElement(By.name("trainingschedulebutton")).click();
    driver.findElement(By.id("bookslot")).click();
    Thread.sleep(2000);
    WebElement selectSport=driver.findElement(By.xpath("//input[@placeholder='Select any sport']"));
    selectSport.click();
    selectSport.sendKeys("Football");
    //actions.sendKeys(Keys.ARROW_DOWN).build().perform();
    actions.sendKeys(selectSport, Keys.ENTER).build().perform();

    WebElement date=driver.findElement(By.id("Date"));
    date.click();
    date.sendKeys("10/23/2024");
    actions.sendKeys(date, Keys.ENTER).build().perform();

    WebElement slot=driver.findElement(By.id("mat-select-value-1"));
    date.click();
    slot.sendKeys("4PM-5PM");
    actions.sendKeys(slot, Keys.ENTER).build().perform();

    driver.findElement(By.name("submitbutton")).click();
}
```

Knowledge Check

Select the correct answers



Which method is mandatory to call in Actions for successful execution of mouse/keyboard event ?

1.build()

2.perform()

3.execute

Knowledge Check(1)

Select the correct answers



Which of the following method is related to mouse event?

1.keyUp()

2.doubleClick()

3.KeyDown()

Knowledge Check(2)

Select the correct answers



Which of the following method is used to right click event?

1.contextClick()

2.doubleClick()

3.moveToElement()

Module Summary

Now, you will be able to:

- Understand about Actions class in WebDriver
- List keyboard and mouse events along with commonly used methods
- Perform automation of mouse events
- Perform automation of keyboard events



Thank You

